



Apostila de **Java Básico**

Manual de consulta rápida — das variáveis às Collections

✓ Compatível com Java 8

Curso de Programação Java

Índice

1. Estrutura de um programa
2. Variáveis e tipos de dados
3. Operadores
4. Entrada e saída de dados
5. Estruturas de decisão (if / switch)
6. Laços de repetição
7. Arrays (vetores e matrizes)
8. Strings
9. Métodos
10. Classes e Objetos (POO)
11. Encapsulamento
12. Herança
13. Polimorfismo
14. Classes abstratas e Interfaces
15. Collections (List, Set, Map)
16. Conversão de tipos (casting)
17. Dicas e boas práticas

1 Estrutura de um programa

Todo programa Java começa pelo método `main`:

```
public class MeuPrograma {  
    public static void main(String[] args) {  
        System.out.println("Ola, mundo!");  
    }  
}
```

- O **nome do arquivo** deve ser igual ao da classe pública: `MeuPrograma.java`.
- Para compilar e executar:

```
javac MeuPrograma.java # gera MeuPrograma.class  
java MeuPrograma      # roda o programa
```

2 Variáveis e tipos de dados

Uma variável guarda um valor. Em Java é preciso dizer **o tipo** dela:

```
int idade = 25;  
double altura = 1.75;  
char sexo = 'M';  
boolean ativo = true;  
String nome = "Ana";
```

Tipos primitivos mais usados

Tipo	Guarda	Exemplo
<code>int</code>	número inteiro	<code>int x = 10;</code>
<code>long</code>	inteiro muito grande	<code>long n = 9999999999L;</code>
<code>double</code>	número com casas decimais	<code>double p = 3.50;</code>
<code>float</code>	decimal (menor precisão)	<code>float f = 3.5f;</code>
<code>char</code>	um único caractere	<code>char c = 'A';</code>
<code>boolean</code>	verdadeiro ou falso	<code>boolean b = true;</code>

💡 **String** não é primitivo (é uma classe), por isso começa com letra maiúscula.

Constantes

Use `final` para um valor que nunca muda:

```
final double PI = 3.14159;
```

3 Operadores

Aritméticos

Operador	Faz	Exemplo (a=10, b=3)	Resultado
<code>+</code>	soma	<code>a + b</code>	<code>13</code>
<code>-</code>	subtração	<code>a - b</code>	<code>7</code>
<code>*</code>	multiplicação	<code>a * b</code>	<code>30</code>
<code>/</code>	divisão	<code>a / b</code>	<code>3</code> (inteira!)
<code>%</code>	resto	<code>a % b</code>	<code>1</code>

⚠ `10 / 3` dá `3` porque os dois são `int`. Para ter `3.33...`, use `double`: `10.0 / 3`.

Relacionais (retornam true/false)

`==` (igual), `!=` (diferente), `>`, `<`, `>=`, `<=`

💡 Para comparar **Strings** use `.equals()`, não `==`: `nome.equals("Ana")`

Lógicos

Operador	Significa	Exemplo
<code>&&</code>	E	<code>idade > 18 && ativo</code>
<code> </code>	OU	<code>a == 1 a == 2</code>
<code>!</code>	NÃO	<code>!ativo</code>

Atribuição e incremento

```
x += 5; // x = x + 5
x -= 2; // x = x - 2
x++;   // x = x + 1
x--;   // x = x - 1
```

4 Entrada e saída de dados

Saída (mostrar na tela)

```
System.out.println("Pula linha no final");
System.out.print("Nao pula linha");
System.out.printf("Nome: %s, Idade: %d%n", nome, idade); // formatado
```

Marcadores do printf: `%s` texto · `%d` inteiro · `%f` decimal · `%.2f` 2 casas · `%c` caractere · `%n` quebra de linha.

Entrada (ler do teclado) com Scanner

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Digite seu nome: ");
String nome = sc.nextLine(); // le uma linha inteira

System.out.print("Digite sua idade: ");
int idade = sc.nextInt();    // le um inteiro
```

Método	Lê
<code>nextLine()</code>	uma linha de texto
<code>next()</code>	uma palavra
<code>nextInt()</code>	um inteiro
<code>nextDouble()</code>	um decimal

💡 **Dica de ouro:** misturar `nextInt()` com `nextLine()` causa bugs (sobra um "Enter"). Uma solução simples é ler tudo com `nextLine()` e converter com `Integer.parseInt(...)` / `Double.parseDouble(...)`.

5 Estruturas de decisão

if / else

```
if (idade >= 18) {
    System.out.println("Maior de idade");
} else if (idade >= 12) {
    System.out.println("Adolescente");
} else {
    System.out.println("Crianca");
}
```

switch

```
switch (opcao) {
    case 1:
        System.out.println("Um");
        break;           // break evita "cair" no proximo case
    case 2:
        System.out.println("Dois");
        break;
    default:
        System.out.println("Opcao invalida");
}
```

6 Laços de repetição

for (quando se sabe quantas vezes)

```
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}
```

while (enquanto a condição for verdadeira)

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

do-while (executa pelo menos uma vez)

```
int opcao;
do {
    // mostra menu e le opcao...
} while (opcao != 0);
```

for-each (percorre coleções/arrays)

```
for (String nome : nomes) {
    System.out.println(nome);
}
```

Controle de fluxo

- **break** → sai do laço imediatamente.
- **continue** → pula para a próxima repetição.

7 Arrays

Um array guarda **vários valores do mesmo tipo** com tamanho fixo.

```
int[] numeros = new int[5];           // 5 posicoes (0 a 4), comecam em 0
numeros[0] = 10;                       // grava na posicao 0
int x = numeros[0];                    // le a posicao 0

double[] notas = {8.5, 7.0, 9.5};     // ja criando com valores
int qtd = notas.length;               // tamanho do array (3)
```

Percorrer

```
for (int i = 0; i < notas.length; i++) {
    System.out.println(notas[i]);
}
// ou com for-each:
for (double nota : notas) {
    System.out.println(nota);
}
```

Matriz (array de duas dimensões)

```
int[][] matriz = new int[3][3];      // 3 linhas x 3 colunas
matriz[0][0] = 1;
```

⚠️ Acessar uma posição que não existe gera erro `ArrayIndexOutOfBoundsException`.

8 Strings

`String` é texto. Alguns métodos muito usados:

```
String s = "Java Basico";

s.length();           // 11 -> quantidade de caracteres
s.toUpperCase();       // "JAVA BASICO"
s.toLowerCase();       // "java basico"
s.charAt(0);           // 'J' -> caractere de uma posicao
s.substring(0, 4);     // "Java"
s.indexOf("Basico");   // 5 -> posicao onde comeca (ou -1)
s.replace("a", "@");   // "J@v@ B@sico"
s.trim();              // remove espacos das pontas
s.isEmpty();          // false -> true se for ""
s.equals("Java");      // false -> compara conteudo
s.equalsIgnoreCase("JAVA BASICO"); // true
s.contains("Java");    // true
s.split(" ");          // ["Java", "Basico"] -> separa em um array
```

💡 Para juntar textos use `+`: `"Ola " + nome`

9 Métodos

Um método é um bloco de código reutilizável.

```
//          tipo de retorno  nome      parametros
public static double calcularMedia(double n1, double n2) {
    return (n1 + n2) / 2; // retorna um valor
}
```

`void` = não retorna nada:

```
public static void exibirMensagem(String texto) {
    System.out.println(texto);
}
```

Chamada:

```
double m = calcularMedia(8.0, 6.0); // m = 7.0
exibirMensagem("Ola!");
```

Sobrecarga (mesmo nome, parâmetros diferentes)

```
static int somar(int a, int b)          { return a + b; }
static int somar(int a, int b, int c) { return a + b + c; }
static double somar(double a, double b) { return a + b; }
```

Escopo

Uma variável criada **dentro** de um método (ou bloco) só existe ali dentro.

10 Classes e Objetos (POO)

Uma **classe** é o molde; um **objeto** é uma instância (cópia real) desse molde.

```
public class Pessoa {  
    String nome;    // atributos (características)  
    int idade;  
  
    // construtor: roda quando criamos o objeto  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;    // this = "este objeto"  
        this.idade = idade;  
    }  
  
    void apresentar() {    // metodo (comportamento)  
        System.out.println("Sou " + nome + ", tenho " + idade);  
    }  
}
```

```
// criando e usando o objeto:  
Pessoa p = new Pessoa("Ana", 30);  
p.apresentar();    // Sou Ana, tenho 30  
System.out.println(p.nome);
```

11 Encapsulamento

Proteger os atributos: deixá-los **private** e dar acesso por métodos **get / set**.

```
public class Conta {  
    private double saldo;    // ninguém altera direto de fora  
  
    public double getSaldo() {    // getter -> le o valor  
        return saldo;  
    }  
  
    public void setSaldo(double saldo) {    // setter -> altera com regra  
        if (saldo >= 0) {  
            this.saldo = saldo;  
        }  
    }  
}
```

- **private** → só a própria classe acessa.
- **public** → qualquer classe acessa.

Benefício: você controla **como** os dados são lidos e alterados (pode validar).

12 Herança

Uma classe pode **herdar** atributos e métodos de outra, com `extends`.

```
public class Animal {
    void comer() {
        System.out.println("Comendo...");
    }
}

public class Cachorro extends Animal {    // Cachorro "é um" Animal
    void latir() {
        System.out.println("Au au!");
    }
}
```

```
Cachorro c = new Cachorro();
c.comer();    // herdado de Animal
c.latir();    // proprio de Cachorro
```

- `super` chama algo da classe-mãe (ex.: `super(nome)` chama o construtor).
- `@Override` indica que estamos reescrevendo um método herdado.

```
public class Gato extends Animal {
    @Override
    void comer() {
        System.out.println("O gato come peixe");
    }
}
```

13 Polimorfismo

"Muitas formas": o mesmo método se comporta diferente conforme o objeto.

```
Animal a1 = new Cachorro();
Animal a2 = new Gato();

a1.comer();    // comportamento do Cachorro
a2.comer();    // comportamento do Gato
```

Mesmo a variável sendo do tipo `Animal`, Java executa o método **do objeto real**. Isso permite guardar tipos diferentes num mesmo array:

```
Animal[] animais = { new Cachorro(), new Gato() };
for (Animal a : animais) {
    a.comer();    // cada um come do seu jeito
}
```

14 Classes abstratas e Interfaces

Classe abstrata

Serve de **base** e não pode ser instanciada. Pode ter métodos prontos e métodos abstratos (sem corpo, que as filhas são obrigadas a implementar).

```
public abstract class Forma {
    abstract double calcularArea(); // sem corpo: cada forma faz do seu jeito

    void descrever() { // metodo normal, ja pronto
        System.out.println("Area: " + calcularArea());
    }
}

public class Circulo extends Forma {
    double raio;
    Circulo(double raio) { this.raio = raio; }

    @Override
    double calcularArea() {
        return 3.14159 * raio * raio;
    }
}
```

Interface

Um "contrato": só diz **o que** a classe deve fazer, sem como. Usa-se `implements`.

```
public interface Imprimivel {
    void imprimir(); // metodos da interface sao abstratos
}

public class Boleto implements Imprimivel {
    @Override
    public void imprimir() {
        System.out.println("Imprimindo boleto...");
    }
}
```

Classe abstrata	Interface
Usa <code>extends</code> (só 1)	Usa <code>implements</code> (pode várias)
Pode ter código pronto	Foca no contrato (o "o quê")
Pode ter atributos comuns	Define o que a classe é capaz de fazer

15 Collections

Estruturas que guardam **vários objetos** e crescem dinamicamente (diferente do array, que tem tamanho fixo). Ficam no pacote `java.util`.

List / ArrayList — lista ordenada, aceita repetidos

```
import java.util.ArrayList;
import java.util.List;

List<String> nomes = new ArrayList<>();
nomes.add("Ana");           // adiciona
nomes.add("Bruno");
nomes.get(0);               // "Ana" -> pega pela posicao
nomes.set(0, "Ana Paula");  // altera a posicao 0
nomes.remove("Bruno");      // remove
nomes.size();               // quantidade
nomes.contains("Ana Paula");// true

for (String nome : nomes) {
    System.out.println(nome);
}
```

Set / HashSet — coleção SEM elementos repetidos

```
import java.util.HashSet;
import java.util.Set;

Set<String> tags = new HashSet<>();
tags.add("java");
tags.add("java");          // ignorado: nao repete
tags.size();               // 1
```

Map / HashMap — pares chave → valor

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> idades = new HashMap<>();
idades.put("Ana", 30);      // adiciona/atualiza
idades.put("Bruno", 25);
idades.get("Ana");          // 30 -> busca pela chave
idades.containsKey("Ana");  // true
idades.remove("Bruno");

for (Map.Entry<String, Integer> e : idades.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

Coleção	Repete?	Ordem	Acesso por
List	sim	mantém ordem de inserção	índice (posição)
Set	não	não garante ordem	só percorrendo
Map	chaves não	não garante ordem	chave

16 Conversão de tipos (casting)

Texto → número

```
int n = Integer.parseInt("42");
double d = Double.parseDouble("3.5");
```

Número → texto

```
String s = String.valueOf(42); // "42"
String s2 = "" + 42;           // "42" (atalho)
```

Entre números

```
double d = 3.9;
int i = (int) d; // 3 (corta as casas decimais, NAO arredonda)

int x = 5;
double y = x; // 5.0 (automatico, sem perda)
```

17 Dicas e boas práticas

- ✓ **Nomes claros:** `idadeUsuario` é melhor que `iu`.
- ✓ **Convenções:** classes em `PascalCase`; variáveis e métodos em `camelCase`; constantes em `MAIUSCULAS`.
- ✓ **Um método, uma tarefa:** métodos curtos e com uma responsabilidade só (alta coesão).
- ✓ **Evite repetir código:** se copiou e colou, provavelmente cabe um método.
- ✓ **Compare Strings com** `.equals()`, nunca com `==`.
- ✓ **Indente o código** para ficar legível.
- ✓ **Comente o "porquê"**, não o óbvio.
- ⚠ Cuidado com **divisão inteira** (`10/3 = 3`) e com **índices fora do array**.

 Esta apostila resume o conteúdo de Java básico do curso. Para aprofundar, consulte os slides de cada aula e pratique com os exercícios.